

MAL: Researching

Checksums, Online Sandboxing, and Virus Total for Malware Research

Blue Team, Malware Analysis, zenmier30

Objectives

- Understand what checksums (hashes) are and why they are a foundational tool in malware identification and file integrity verification.
- Learn the most common hashing algorithms used in malware analysis: MD5, SHA-1, SHA-256, and SHA-512, and understand the differences between them.
- Understand hash collisions, what they mean for the reliability of an algorithm, and why the malware community continues to use MD5 alongside stronger algorithms.
- Generate checksums using two methods on a Windows machine: PowerShell's Get-FileHash command and the CertUtil command-line tool.
- Understand how online sandboxes work, how to read a sandbox analysis report, and what kind of behavioral information they provide about an unknown file.
- Use Virus Total to submit file hashes and determine whether a sample has been previously identified as malicious by the security community.

Tools Used

Tool	Purpose
PowerShell, Get-FileHash	Built-in Windows PowerShell command is used to compute MD5 and SHA256 hashes of any file. Syntax: Get-FileHash .\filename.exe -Algorithm MD5. No additional software required.
CertUtil	Windows built-in certificate management tool that can also compute file hashes. Syntax: certutil -hashfile TryHackMe.exe SHA256. Commonly seen in both legitimate admin work and LOL attacks.
HashCheck (File Properties)	Shell extension is installed lablab VM that adds a File Hashes tab to the standard Windows file Properties dialog. Allows hash retrieval by right clicking any file without opening a terminal.
Hybrid-Analysis.com	Free online sandbox platform that runs submitted files in an isolated environment and generates a detailed report covering static indicators, process behavior, network traffic, DNS queries, and dropped files.
VirusTotal	Free online threat intelligence platform. Accepts file uploads, URLs, IP addresses, or hashes and checks them against 70+ antivirus engines. Confirms whether a specific hash has been previously identified as malicious.

Checksums 101

A checksum, also called a hash, is a fixed-length string generated by running a file through a mathematical algorithm. The same file always produces the same hash. Even a single changed byte in the file produces a completely different hash output.

This property makes checksums one of the most reliable ways to uniquely identify a file. In malware analysis, hash acts as a fingerprint: instead of sharing a dangerous file, analysts share its hash, which can then be searched on platforms like Virus Total to find existing analysis reports.

Hashing Algorithms

Algorithm	Output Length	Notes
MD5	128-bit (32 hex chars)	Developed by Ronald Rivest. Fast and widely used in the malware community despite known collision vulnerabilities. Still valuable for quick lookups.
SHA-1	160-bit (40 hex chars)	Stronger than MD5 but also considered weak against deliberate collision attacks. Still used in legacy systems.
SHA-256	256-bit (64 hex chars)	Current standard for secure file verification. Highly resistant to collisions. Recommendation for all new malware analysis workflows.
SHA-512	512-bit (128 hex chars)	The next algorithm in the SHA-2 series after SHA-256. Even stronger output but less commonly used in day-to-day malware analysis.

Hash Collisions

A hash collision occurs when two different files produce the same hash output. For MD5, a collision can theoretically be engineered by hashing billions of files: the probability becomes significant after approximately 6 million (or more precisely, billions of) files are hashed.

Despite this weakness, MD5 remains widely used in the malware community because its purpose is identification, not security. If a file matches a known bad MD5 hash in a threat intelligence database, that is strong evidence of a known threat, even if MD5 itself is not cryptographically robust for authentication purposes.

Online Sandboxing

A sandbox is an isolated virtual environment that runs an unknown file and records everything it does: which processes it launches, which files it creates or modifies, what network connections it makes, and what registry keys it touches. None of this activity affects the real machine because the sandbox is completely separated from it.

Online sandboxes like Hybrid-Analysis.com automate this entire process. The analyst submits a file, waits for the analysis to complete (typically 5 to 15 minutes), and receives a structured report. Because the hash is recorded, future analysts searching for the same sample will immediately find the existing report, removing the need to reanalyze already-known threats.

Reading a Sandbox Report

The Emotet malware family was used as a case study in this task. Emotet is classified as a Trojan, specifically one that spreads through malicious email attachments. A Hybrid-Analysis report for an Emotet sample produced the following sections:

- Indicators and Context: High-level verdict, threat score, and malware family classification. For Emotet: Threat type listed as Trojan, delivery method listed as malicious email attachments.
- Static Analysis Summary: File metadata, hash values, PE header details, imported libraries, and embedded strings, all without executing the file.
- Dynamic Behavior Timeline: A chronological list of every action the malware took when run in the sandbox, including process creation, file writes, and system modifications.
- Process Execution Tree: A visual map showing which process launched which child process, making it easy to see the full execution chain from the initial sample.

- Network Analysis: All DNS queries made, IP addresses contacted, HTTP requests sent, and any data that was transmitted, revealing C2 server addresses and exfiltration targets.
- Extracted Strings and Dropped Files: Readable content pulled from memory during execution and any secondary files written to disk by the malware.

03/06/2021
c378387344e0a552dc065de6bfa607fd26e0b5c569751c79fb9c6f2e91c9807 | ANY.RUN - Free Malware Sandbox Online



Interactive malware hunting service

General Info

File name
Full analysis
Verdict
Threats:

emotet.doc

Malicious activity

Emotet

Emotet is one of the most dangerous trojans ever created. Over the course of its lifetime, it was upgraded to become a very destructive malware. It targets mostly corporate victims but even private users get infected in mass spam email campaigns.

Malware Trends Tracker
More details

Analysis date
OS:
Tags:
Indicators:

9/16/2019, 13:54:48
Windows 7 Professional Service Pack 1 (build: 7601, 32 bit)
macros macros-on-open emotet-doc emotet generated-doc trojan loader


MIME:
File info:
MD5
SHA1
SHA256
SSDEEP

application/vnd.openxmlformats-officedocument.wordprocessingml.document
Microsoft Word 2007+
B92021CA10AED3046FC3BE5AC1C2A094
0FB1AD5B53CDD09A7268C823EC796A6E623F086F
C378387344E0A552DC065DE6BFA607FD26E0B5C569751C79FB9C6F2E91C9807
3072: /MSKNOK2ER/YR5DPQKAJNDU1CKBWN0PQJFWSQ:ZKOROKDPQZQQKMN0SCR

TAKE YOUR SECURITY
TO THE NEXT LEVEL

- ✓ Realtime interaction
- ✓ Process monitoring
- ✓ Network tracking
- ✓ Inspect behavior graph
- ✓ IOC gathering

JOIN FREE!

with [ANY.RUN](#) Community Version

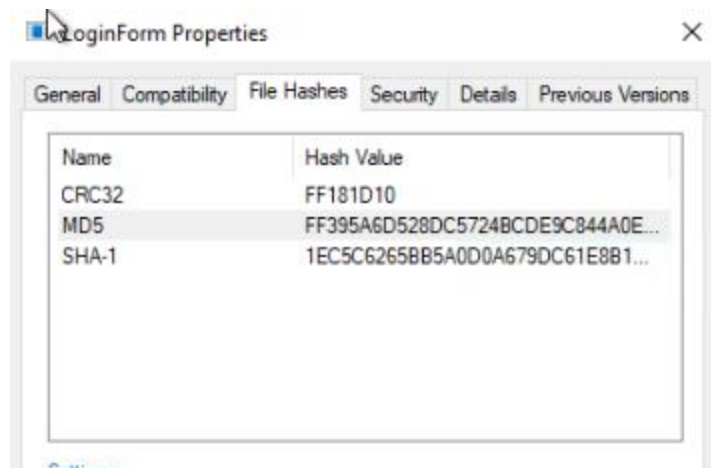
Calculating and Reporting Checksums

The lab machine (Windows, accessible via RDP) contains sample files on the Administrator desktop used for the checksum exercises. Two methods were demonstrated for computing checksums, covering both GUI and command-line approaches.

Method 1, File Properties (HashCheck GUI)

A HashCheck shell extension installed on the lab VM adds a File Hashes tab to the standard Windows Properties dialog. To use it: right-click the target file, select Properties, and navigate to the File Hashes tab. The MD5, SHA-1, and SHA-256 hashes are displayed immediately without opening any terminal.

This method is convenient for quick one-off lookups but is not practical for processing multiple files at once. For batch hashing, the command-line methods below are used instead.



Method 2, PowerShell (Get-FileHash)

PowerShell's built-in Get-FileHash command computes hashes for any file without requiring additional software. Two common syntax examples used during the practical:

```
Get-FileHash .\TryHackMe.exe -Algorithm MD5
Get-FileHash .\TryHackMe.exe -Algorithm SHA256
```

To hash all files in a directory at once, the wildcard syntax Get-FileHash .* -Algorithm MD5 was used. This is useful when given a folder of unknown files to triage quickly.

```
PS C:\Users\Administrator\Desktop> Get-FileHash TryHackMe.exe -Algorithm sha256
```

Algorithm	Hash	Path
SHA256	6F870C80361062E8631282D31A16872835F7962222457730BC55676A61AD1EE0	C:\Users\Admini...

Method 3, CertUtil

The CertUtil tool provides an alternative hash calculation method and is notable because it is also a known LOL (Living Off the Land) tool. Its -hashfile flag computes file hashes from any Windows command prompt or PowerShell terminal:

```
certutil -hashfile TryHackMe.exe SHA256
```

The output includes the hash algorithm name followed by the computed hash on the next line. Recognising this command in a log is important: legitimate use by an analyst looks identical to an attacker using CertUtil to fingerprint files on a compromised machine.

```
PS C:\Users\Administrator\Desktop> CertUtil -hashfile TryHackMe.exe sha256
SHA256 hash of TryHackMe.exe:
6f870c80361062e8631282d31a16872835f7962222457730bc55676a61ad1ee0
CertUtil: -hashfile command completed successfully.
PS C:\Users\Administrator\Desktop>
```

Virus Total

Virus Total is a free online platform that checks submitted files, URLs, IP addresses, or hash values against the detection engines of 70 or more antivirus and security vendors simultaneously. It returns a verdict for each engine, a community score, and a detailed report covering file metadata, behavior indicators, and associated threat intelligence.

In malware analysis, submitting a file hash rather than the file itself is the preferred approach for known samples. If the hash has been seen before, Virus Total will return the full analysis report instantly. This avoids uploading a potentially sensitive or classified malware sample to an external platform.

Practical Exercise

The MD5 hashes computed in Task 5 were submitted to Virus Total to determine whether each sample was previously identified as malicious. The three files used (aws.exe, Netlogo.exe, and vlc.exe) were checked individually. All three returned Nay (not detected as malicious) across the vendor engines, demonstrating that a clean Virus Total result does not guarantee a file is safe, only that it matches no known signatures at the time of the search.

This is an important limitation to understand: new or modified malware that has not yet been submitted to Virus Total will return a clean result. Zero-day or custom-built malware specifically avoids detection by staying off known hash databases. A clean result lowers suspicion but does not close an investigation.

0

Community Score

4

No security vendors flagged this file as malicious

6f870c80361062e8631282d31a16872835f796222457730bc55676a61ad1ee0

Size: 6.47 MB

Last Analysis Date: 8 days ago

EXE

aws.exe

checks user input

runtime modules

detect debug environment

assembly

direct cpu clock access

64bits

DETECTION

DETAILS

RELATIONS

BEHAVIOR

COMMUNITY

Join our Community and enjoy additional community insights and crowdsourced detections, plus an API key to automate checks.

Basic properties

MD5

18e07a22126d8ee7af6a23727060edb

SHA-1

549d8bd241f8d15a32d26396c5cde8b03061

SHA-256

6f870c80361062e8631282d31a16872835f796222457730bc55676a61ad1ee0

Vhash

06d905d1cd1cd15d61d2309d07c00a7061d100110c5d8d300e8030d8e

Authenthash

aaacfb807beb172113d643626af308825292e073765c44b3f5bf8bea736asca

ImpHash

0617b42806cd39d19867aa9e0af6d28f

SSDEEP

49152:MRsl:uSj:JmfrRein2VGS:11aWiqz:NNVofJclVqLL7gl:RUWNSFoSoDEjdfLLDNVHo

TLSH

TJBB8637772A867CC56F827AC497C890E93380B51B37CEB426012695F32AC45E3E760

File type

Win32 EXE (executable) (desktop) (windows) (pe) (peexe)

Magic

PE32+ executable (GUI) x86-64, for MS Windows

TnD

Win64 Executable (generic) (51.8%) OS/2 Executable (generic) (16.1%) Generic Win/DOS Executable (15.9%) DOS Executable (generic) (15.8%) VXD Driver (0.2%)

DetectItEasy

PE64 Compiler: Embarcadero Delphi (XE3 X4) [Professional] | Linker: Turbo Linker (8.0) [GUI64]

Magika

PEBIN

File size

6.47 MB (6789120 bytes)

History

Creation Time

2020-02-28 11:16:36 UTC

First Submission

2020-05-21 10:29:58 UTC

Last Submission

2025-10-07 18:24:53 UTC

Last Analysis

2026-05-29 03:22:58 UTC

Investigation Methodology

The following sequence represents the standard file research workflow practiced across this lab.

Step	Action	Detail
1	Compute Hash	Run Get-FileHash or certutil on the suspicious file. Record both MD5 and SHA256 to support lookups across all major platforms.
2	Virus Total Lookup	Search the hash on Virus Total. If previously seen, review the full report for classification, behavior indicators, and linked infrastructure.

3	Sandbox Submission	If the hash is unknown or the sample is new, submit to Hybrid-Analysis. Wait for the report covering process tree, network activity, and dropped files.
4	Analyze the Report	Read the sandbox report sections in order: verdict, static summary, process tree, network connections, extracted strings. Extract all IOCs.
5	Document and Share	Record the hash, classification, IOCs, and sandbox URL. Share the hash with the team rather than the file to avoid passing a live sample.

DETECTION OPPORTUNITIES

- Alert on CertUtil executed with -hashfile or -urlcache flags from a non-admin user account or from an unexpected parent process, as this may indicate a LOL attack using the tool to fingerprint or download files.
- Use automated hash lookups as part of the alert triage pipeline: any new executable appearing on an endpoint should have hash checked against Virus Total before manual analysis begins.
- A file returning a clean Virus Total result but exhibiting suspicious behavior in a sandbox (outbound connections, process injection, registry persistence) should be escalated regardless of the vendor score.
- Hash the same file at different times: if the hash changes between scans, the file has been modified, which is itself an indicator of tampering or active malware dropping a modified copy.

Summary

This lab established the core research skills that underpin all malware analysis work: generating file hashes, searching those hashes on threat intelligence platforms, and reading sandbox reports to understand unknown file behavior.

Checksums are the universal language of malware identification. A SHA256 hash uniquely identified a file and allows analysts worldwide to share findings without sharing the dangerous file itself. Virus Total and sandbox platforms like Hybrid-Analysis extend that single hash into a rich body of threat intelligence.

The key limitation reinforced by this lab is that a clean Virus Total result does not mean a file is safe. Unknown, modified, or purpose-built malware evades hash-based detection by design. Hashing is the first step, not the last.

Area Covered	Key Takeaway
Checksums 101	MD5, SHA-1, SHA-256, and SHA-512 covered. MD5 developer: Ronald Rivest. SHA-512 follows SHA-256 in the series. Hash collisions require billions of files for MD5.
Hash Generation	Three methods practiced: PowerShell Get-FileHash, CertUtil -hashfile, and the HashCheck GUI in Windows file Properties.
Online Sandboxing	Hybrid-Analysis.com used as case study with Emotet (Trojan, spreads via malicious email). Report sections: verdict, static, process tree, network, strings.
Virus Total	MD5 hashes of aws.exe, Netlogo.exe, vlc.exe submitted. All returned Nay. Clean results do not confirm safety for unknown or modified samples.

Key Limitation

Hash-based detection only works for known samples. New or modified malware bypasses Virus Total lookup. Always follow up with sandbox analysis.
